Paper 3 — Foundations for Structural Analysis

Jose Luna

July 6, 2026

Abstract

Paper 1 formalized workload-relative dependency. Paper 2 established a deterministic compilation framework that constructs canonical structural objects and workload-specific analysis graphs. This paper addresses the next foundational question: what constitutes a well-defined structural analysis over canonical structural objects?

We develop a representation-invariant framework for structural analysis, classify analysis families, define structural analysis operators, and establish mathematical properties required for analyses to remain independent of representational artifacts.

1 Introduction

This paper continues the series by asking a single foundational question:

What is a structural analysis?

We take as given that Paper 2 produces canonical structural objects that factor out representational artifacts while preserving the structural content relevant to a workload class. Paper 3 asks what it means for an analysis to be *well defined* on such canonical objects, and what kinds of analysis families can be expressed without re-introducing representation dependence.

2 Background

2.1 Paper 1 (workload-relative dependency)

Paper 1 defines a workload-relative dependency predicate evaluated on a directed analysis graph, operationalized via counterfactual deletion and reachability. In Paper 3, this serves as a motivating example of an analysis whose output can be stated both as a Boolean answer (dependent / not dependent) and as a witness object (e.g., a path that survives deletion, or a cut set that blocks reachability).

2.2 Paper 2 (canonical structural objects and compilation)

Paper 2 defines a deterministic compilation framework that maps heterogeneous system descriptions into canonical structural objects, and then projects those objects into workload-specific analysis graphs. Paper 3 treats this as an interface contract: analyses should be formulated over the canonical object (or its workload-specific projections) in a way that is invariant to representation artifacts such as naming, ordering, and serialization choices.

3 Related Work

This section surveys the main bodies of work that inform a representation-invariant notion of structural analysis. The emphasis is on *analysis families* and *interfaces* (what an analysis consumes and produces), rather than on any single domain-specific graph formalism.

3.1 Graph algorithms

Classical graph algorithms supply the core computational primitives used by many structural analyses, including traversal and reachability, shortest paths, and cut/flow-based separations. These primitives ground the "navigation" and "vulnerability" families in the taxonomy of Section 4. Standard algorithmic treatments include Dijkstra's shortest path algorithm and related graph optimization techniques, as well as max-flow/min-cut dualities that underpin separation-style analyses [10, 1].

3.2 Graph query languages

Graph query languages make structural analyses programmable and compositional by separating *what* is asked from *how* it is computed. Two major lines are (i) RDF/SPARQL-style query languages for labeled, schema-aware graphs [14] and (ii) property-graph query languages (e.g., Cypher) that emphasize pattern matching with attributes [3]. Datalog and its descendants provide a unifying logic-programming substrate for recursive graph queries and fixpoint computations, connecting naturally to the recursive analysis family in Section 4 [5].

3.3 Abstract interpretation

Abstract interpretation formalizes static analysis as computation over sound abstractions, typically organized as lattices with monotone transfer functions and fixpoints [7]. While Paper 3 does not introduce theorems yet, the abstract-interpretation viewpoint motivates two design requirements for structural analyses: (i) an explicit statement of the abstraction boundary (what is forgotten) and (ii) invariance with respect to representation-level differences that lie below that boundary.

3.4 Program analysis

Program analysis provides mature examples of structural analyses over compiler-derived graphs and intermediate representations, including control-flow graphs, SSA form, dependence graphs, and slicing [15, 8, 12, 17]. These works show how analysis results can be phrased as (a) predicates, (b) sets of program elements, and (c) witnesses (e.g., dependence edges or slice criteria) computed over canonicalized representations.

3.5 Provenance and workflow models

Provenance research treats explanation as a first-class analysis goal: instead of returning only an answer, the analysis returns a structured justification of how an output relates to inputs and transformations [4, 6, 13, 9]. This aligns with the "explanatory/provenance" family in Section 4 and motivates witness objects that are subgraphs, derivation traces, or minimal explanations.

Family	Typical question / operator	Typical output
Reachability / navigation	Can t be reached from R under relation family Rel?	Boolean; path witness
Separation / vulnerability	What removals block reachability (cuts, separators)?	Cut set; minimal witness
Data-flow / dependence	What information flows or depends on what (def–use, PDG-style)?	Dependence relation; slice
Recursive / fixpoint	What is the closure under rules (transitive closure, recursive queries)?	Fixpoint set / relation
Quantitative criticality	How critical is component x (centrality, flow impact, sensitivity)?	Score; ranking
Counterfactual	How do answers change under perturbations Δ?	Delta in answers; witness
Explanatory / provenance	Why did y occur / what contributed?	Explanation subgraph / trace
Transformational	What rewrite produces a desired property (refactoring, normalization, repair)?	Transformation plan / script

Table 1: Taxonomy of structural analysis families (research synthesis categories) used to organize Paper 3.

3.6 Resilience and network science

Network science studies how structural properties of graphs respond to failures, attacks, and perturbations. Robustness and fragility analyses often quantify the effect of removing vertices/edges on connectivity, reachability, or flow, and characterize critical components via centrality and related measures [2, 16]. This literature informs the perturbation and criticality families in Section 4.

3.7 Graph transformation

Graph transformation theory provides formal languages for specifying and reasoning about structural rewrites (e.g., deletion, addition, relabeling, and rule-based rewiring). Double-pushout (DPO) and related approaches treat transformations as typed, compositional rewrite rules [11]. This viewpoint motivates treating "counterfactual" and "transformational" analyses as operators indexed by explicit perturbation families.

4 Analysis Taxonomy

Table 1 summarizes a taxonomy of structural analysis families used in this paper. The goal is not to be exhaustive, but to provide a stable set of categories for (i) positioning new analyses and (ii) stating representation-invariance requirements in Section 5.

4.1 Expressive coverage of the analysis interface

We now give representative instantiations of each analysis family in Table 1 using only the interface objects introduced in Section 5. Each instantiation follows the same template to make the codomain (Y) and optional witness/perturbation structure explicit.

Reachability / navigation.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** $\{\mathsf{true}, \mathsf{false}\}$.
- **Witness (optional):** Ev contains paths (or traversal traces) witnessing reachability.
- **Perturbation (optional):** none.
- **Remarks:** Navigation analyses are observables whose codomain is Boolean and whose witnesses are paths.

Separation / vulnerability.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** sets of components (e.g., cut sets / separators).
- **Witness (optional):** Ev contains certificates such as (i) a cut set that blocks reachability and/or (ii) a witness that smaller sets do not.
- **Perturbation (optional):** Δ can be taken as a deletion family with $\delta \in \Delta$ removing a candidate set.
- **Remarks:** Vulnerability analyses fit as observables that return separating sets, optionally parameterized by deletion perturbations.

Data-flow / dependence.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** a relation (e.g., dependence pairs) or a set-valued slice.
- **Witness (optional):** Ev contains dependence edges, def–use chains, or a slice explanation subgraph.
- **Perturbation (optional):** none.
- **Remarks:** Dependence analyses are observables with relational/set codomains and graph-structured witnesses.

Recursive / fixpoint.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** a least/greatest fixpoint set or relation.
- **Witness (optional):** Ev contains derivation traces or proof trees for membership/non-membership.
- **Perturbation (optional):** none.
- **Remarks:** Recursive analyses fit as observables whose outputs are fixpoints, with derivation-style witnesses.

Quantitative criticality.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** real-valued scores, vectors, or rankings.
- **Witness (optional):** Ev contains computation certificates (e.g., flow/cut certificates) or sensitivity explanations.
- **Perturbation (optional):** Δ may encode failures/attacks used to define marginal contribution or sensitivity.
- **Remarks:** Criticality analyses are observables with quantitative codomains, often defined relative to perturbation families.

Counterfactual.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** deltas between observations (e.g., $\mathcal{O}(\Sigma)$ vs. $\mathcal{O}(\delta(\Sigma))$), or a summary over Δ.
- **Witness (optional):** Ev contains the perturbation δ plus a certificate of change (or invariance).
- **Perturbation (optional):** a required perturbation family Δ.
- **Remarks:** Counterfactual analyses are structural analysis operators indexed by perturbations and returning differences.

Explanatory / provenance.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$.
- **Output (Y):** an explanation object (e.g., a subgraph, trace, or provenance record).
- **Witness (optional):** Ev may coincide with the output (the explanation is itself the witness).
- **Perturbation (optional):** none.
- **Remarks:** Explanatory analyses fit by taking Y to be a structured witness object rather than a scalar answer.

Transformational.

- **Canonical object:** Σ.
- **Observable:** $\mathcal{O} : \Sigma \to Y$ with $Y \in \mathsf{Obs}$ (often a "goal test" observable).
- **Output (Y):** a transformation plan (e.g., a sequence of perturbations) or a rewritten object.
- **Witness (optional):** Ev contains the plan plus a check that the transformed object satisfies the goal.
- **Perturbation (optional):** Δ is the space of admissible rewrites; plans are elements of Δ^*.
- **Remarks:** Transformational analyses fit as operators that return admissible rewrite plans drawn from a perturbation family.

These instantiations are illustrative rather than exhaustive. Their purpose is to demonstrate that the proposed interface accommodates the principal classes of structural analyses identified in the literature survey. The absence of an instantiation would indicate a limitation of the interface and motivate refinement before any general theorems are proposed.

5 Definitions

This section introduces the core objects used throughout Paper 3. No theorems are stated in v0.2; the goal is to fix interfaces and terminology.

The following definitions specify the minimal interface required for structural analyses over canonical structural objects. They are intentionally independent of any particular analysis family, algorithm, or application domain so that the interface can be validated across the taxonomy introduced in Section 4 before theorem-level properties are established.

Definition 1 (Admissible observation spaces)**.** *Let* Obs *denote a collection of* admissible observation spaces*. An element* $Y \in \mathsf{Obs}$ *is a (typed) codomain of observations (e.g., Booleans, sets, rankings, rewritten structures, provenance objects).*

Definition 2 (Structural observable)**.** *Let* Σ *denote a canonical structural object (as produced by Paper 2). A* structural observable *is a deterministic map* $\mathcal{O}$ *together with a chosen observation space* $Y \in \mathsf{Obs}$ *such that*

$$\mathcal{O} : \Sigma \to Y.$$

Definition 3 (Representation-invariant observable)**.** *Fix an equivalence relation $\equiv$ on canonical structural objects that identifies representational artifacts (e.g., naming and ordering). A structural observable $\mathcal{O}$ is* representation invariant *(with respect to $\equiv$) if for all Σ_1, Σ_2,*

$$\Sigma_1 \equiv \Sigma_2 \implies \mathcal{O}(\Sigma_1) = \mathcal{O}(\Sigma_2).$$

Definition 4 (Evidence / witness object)**.** *Fix an observation space $Y \in \mathsf{Obs}$. An* evidence object *(or* witness*) for an observable value $y \in Y$ is an auxiliary artifact $e \in \mathsf{Ev}$ such that e can be mechanically checked (relative to Σ) to justify y. Typical witnesses include paths, cut sets, subgraphs, derivation traces, slices, or certificates for quantitative scores.*

Definition 5 (Structural analysis operator)**.** *A* structural analysis operator *is a deterministic operator A that maps a canonical structural object to an observable value in some $Y \in \mathsf{Obs}$,*

$$\mathsf{A} : \Sigma \to Y.$$

Some operator classes additionally produce evidence; in that case, one may write $\mathsf{A}(\Sigma) = (y, e)$ with $y \in Y$ and $e \in \mathsf{Ev}$.

Definition 6 (Perturbation family)**.** *A* perturbation family *is a set Δ of deterministic transformations on canonical structural objects,*

$$\delta : \Sigma \to \Sigma \qquad (\delta \in \Delta),$$

intended to model counterfactual edits such as deletions, additions, rewiring, abstraction changes, or policy constraints.

6 Canonical Structural Objects Revisited

7 Structural Analysis

8 Structural Analysis Operators

9 Perturbation-Based Structural Analyses

10 Properties of Structural Analyses

11 Discussion

12 Future Work

Future work will (i) prove general properties of admissible structural analyses and (ii) instantiate the framework on concrete analysis families.

On the theory side, promising directions include richer classes of admissible observation spaces (e.g., products, function spaces, lattices, and probabilistic observation spaces), temporal and streaming variants of structural analysis, and distributed/parallel formulations.

On the application side, future work will instantiate the framework to families such as resilience, redundancy, blast radius, counterfactual reasoning, provenance/explanation, and optimization-oriented transformational analyses, supported by empirical case studies, tool support, and (where appropriate) governance and assurance applications.

13 Conclusion

A Notation (alphabetical)

Symbol	Meaning
Σ	Canonical structural object (as produced by Paper 2)
$\equiv$	Representation-artifact equivalence on canonical structural objects
Obs	Collection of admissible (typed) observation spaces (Definition 1)
Y	A chosen observation space with $Y \in \mathsf{Obs}$
Ev	Evidence (witness) space for checkable justifications
$\mathcal{O}$	Structural observable $\Sigma \to Y$ with $Y \in \mathsf{Obs}$ (Definition 2)
A	Structural analysis operator (Definition 5)
e	Evidence / witness object (Definition 4)
Δ	Perturbation family (Definition 6)
δ	A perturbation transformation $\Sigma \to \Sigma$ with $\delta \in \Delta$
R, t	Example anchor sets/targets for navigation-style analyses
S	Example candidate set for deletion/separation-style analyses

Table 2: Core notation used throughout the manuscript, alphabetized by symbol.

B Appendix A

C Appendix B

References

[1] Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin. *Network Flows: Theory, Algorithms, and Applications.* Prentice Hall, 1993.

[2] Réka Albert, Hawoong Jeong, and Albert-László Barabási. Error and attack tolerance of complex networks. *Nature*, 406(6794):378–382, 2000.

[3] Renzo Angles, Marcelo Arenas, Pablo Barceló, Aidan Hogan, Juan L. Reutter, and Domagoj Vrgoc. Foundations of modern query languages for graph databases. *ACM Computing Surveys*, 50(5):68:1–68:40, 2018.

[4] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. *Proceedings of the 8th International Conference on Database Theory (ICDT)*, pages 316–330, 2001.

[5] Stefano Ceri, Georg Gottlob, and Letizia Tanca. *Logic Programming and Databases.* Springer, 1990.

[6] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.

[7] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977.

[8] Ron Cytron, Jeanne Ferrante, Barry K. Rosen, Mark N. Wegman, and F. Kenneth Zadeck. Efficiently computing static single assignment form and the control dependence graph. In *Proceedings of the 16th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 25–35, 1991.

[9] Ewa Deelman, Dennis Gannon, Matthew Shields, and Ian Taylor. Workflows and e-science: An overview of workflow system features and capabilities. *Future Generation Computer Systems*, 25(5):528–540, 2009.

[10] E. W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.

[11] Hartmut Ehrig, Karsten Ehrig, Ulrike Prange, and Gabriele Taentzer. *Fundamentals of Algebraic Graph Transformation*. Springer, 2006.

[12] Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[13] Paul Groth and Luc Moreau. PROV-Overview: An overview of the PROV family of documents. W3c recommendation, W3C, April 2013.

[14] Steve Harris, Andy Seaborne, and Eric Prud'hommeaux. SPARQL 1.1 query language. W3c recommendation, W3C, March 2013.

[15] Thomas Lengauer and Robert Endre Tarjan. A fast algorithm for finding dominators in a flowgraph. *ACM Transactions on Programming Languages and Systems*, 1(1):121–141, 1979.

[16] Mark Newman. *Networks: An Introduction*. Oxford University Press, 2010.

[17] Mark Weiser. Program slicing. In *Proceedings of the 5th International Conference on Software Engineering (ICSE)*, pages 439–449, 1981.